

```

/**
 * Reference API layer for the Ferry Cargo Log app, backed by Neon Postgres.
 *
 * Contract expected by the front-end (cargo-ferry-log.html):
 *   GET  /api/data/:key    -> { value: <json or null> }
 *   PUT  /api/data/:key    body { value: <json> } -> upserts, returns { value }
 *
 * This example uses Express + the Neon serverless driver (HTTP-based, works
 * anywhere - Node, Vercel Functions, Cloudflare Workers with small tweaks).
 * Swap the Express wrapper for your platform's handler signature as needed;
 * the Neon query logic in the middle stays the same.
 *
 * Setup:
 *   npm install express @neondatabase/serverless cors
 *   export DATABASE_URL="postgresql://<user>:<password>@<host>/<db>?sslmode=requ
 *   node server.js
 */

const express = require('express');
const cors = require('cors');
const crypto = require('crypto');
const nodemailer = require('nodemailer');
const twilio = require('twilio');
const { authenticator } = require('otplib');
const { neon } = require('@neondatabase/serverless');

const sql = neon(process.env.DATABASE_URL);
const app = express();

// The default express.json() body limit is 100kb. Shipments now carry
// compressed photos (base64) inside the same JSON blob, and even a handful
// of them across different shipments comfortably exceeds that -- once it
// does, every save silently starts failing with a 413. 25mb gives plenty of
// headroom for many photos in a single save.
app.use(express.json({ limit: '25mb' }));

// --- Owner PIN reset (emailed via Gmail) -----
// GMAIL_USER / GMAIL_APP_PASSWORD are the credentials the app itself sends
// FROM (a Gmail account with an "App Password" generated at
// https://myaccount.google.com/apppasswords - this requires 2-Step
// Verification to be on). The reset link is sent TO whatever address the
// Owner has registered in Settings ("Registered Gmail"). APP_URL is the
// public URL of the front-end (your GitHub Pages site), used to build the
// link, e.g. https://YOUR-USERNAME.github.io

```

```

const GMAIL_USER = process.env.GMAIL_USER;
const GMAIL_APP_PASSWORD = process.env.GMAIL_APP_PASSWORD;
const APP_URL = (process.env.APP_URL || '').replace(/\/+$/, '');
const RESET_TOKEN_TTL_MS = 30 * 60 * 1000; // 30 minutes

const mailer = (GMAIL_USER && GMAIL_APP_PASSWORD)
  ? nodemailer.createTransport({
      service: 'gmail',
      auth: { user: GMAIL_USER, pass: GMAIL_APP_PASSWORD },
    })
  : null;

// --- Owner PIN reset (texted via Twilio SMS) -----
// TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN come from your Twilio console.
// TWILIO_PHONE_NUMBER is the Twilio number the text is sent FROM (must be
// SMS-capable). The reset link is texted TO whatever number the Owner has
// registered in Settings ("Registered Phone").
const TWILIO_ACCOUNT_SID = process.env.TWILIO_ACCOUNT_SID;
const TWILIO_AUTH_TOKEN = process.env.TWILIO_AUTH_TOKEN;
const TWILIO_PHONE_NUMBER = process.env.TWILIO_PHONE_NUMBER;

const smsClient = (TWILIO_ACCOUNT_SID && TWILIO_AUTH_TOKEN)
  ? twilio(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
  : null;

// --- Owner PIN reset (authenticator app code) -----
// TOTP_SECRET is a base32 secret known only to this server (set as a Render
// env var, never returned by any API response) and to whatever authenticator
// app (OneAuth, Google Authenticator, Authy, etc.) the Owner entered it into.
// Unlike the email/phone paths, this needs no outside service to work -- the
// 6-digit code the app shows is verified locally against the same secret.
const TOTP_SECRET = process.env.TOTP_SECRET;
authenticator.options = { window: 1 }; // allow 1 step (\u00b130s) of clock drift

// Restrict this to your actual GitHub Pages origin in production,
// e.g. cors({ origin: 'https://bakku3344.github.io' })
app.use(cors());

// Never let browsers or intermediate CDNs cache these responses -- this data
// changes constantly (new check-ins, staff PIN changes, etc), and a cached
// stale response can make the app look broken even when the DB is fine.
app.use((req, res, next) => {
  res.set('Cache-Control', 'no-store, no-cache, must-revalidate, proxy-revalidate');
  res.set('Pragma', 'no-cache');
  res.set('Expires', '0');
  next();
});

```

```

const ALLOWED_KEYS = new Set(['shipments', 'rates', 'settings', 'trips']);

// Phase 3: every shipments/rates/settings/trips read & write is now scoped
// to a specific boat_id, so each boat's dispatch data is private to it.
// boatId isn't re-verified against a passkey here -- same low-auth model as
// the rest of this API (see README) -- but it does have to be a real,
// existing boat, which at least rules out typos/garbage IDs silently
// creating orphaned data.
// Simple public lookup of a boat's display name -- used client-side to
// build things like auto-generated staff usernames (name.boatname).
app.get('/api/boats/:id/name', async (req, res) => {
  try {
    const rows = await sql`SELECT name FROM boats WHERE id = ${req.params.id}`;
    if (!rows.length) return res.status(404).json({ error: 'unknown boat' });
    res.json({ name: rows[0].name });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: 'db error' });
  }
});

app.get('/api/data/:boatId/:key', async (req, res) => {
  const { boatId, key } = req.params;
  if (!ALLOWED_KEYS.has(key)) return res.status(400).json({ error: 'unknown key' });
  try {
    const rows = await sql`SELECT value FROM app_data WHERE boat_id = ${boatId}`;
    res.json({ value: rows.length ? rows[0].value : null });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: 'db error' });
  }
});

app.put('/api/data/:boatId/:key', async (req, res) => {
  const { boatId, key } = req.params;
  if (!ALLOWED_KEYS.has(key)) return res.status(400).json({ error: 'unknown key' });
  const { value } = req.body || {};
  if (value === undefined) return res.status(400).json({ error: 'missing value' });
  try {
    const boatRows = await sql`SELECT id FROM boats WHERE id = ${boatId}`;
    if (boatRows.length === 0) return res.status(404).json({ error: 'unknown boat' });
    await sql`
      INSERT INTO app_data (boat_id, key, value, updated_at)
      VALUES (${boatId}, ${key}, ${JSON.stringify(value)}::jsonb, now())
      ON CONFLICT (boat_id, key) DO UPDATE SET value = ${JSON.stringify(value)}::
    `;
  }
});

```

```

    res.json({ value });
  } catch (e) {
    console.error(e);
    res.status(500).json({ error: 'db error' });
  }
});

// Request a reset link. Always responds the same way regardless of whether
// the email matches, whether Gmail is configured, or whether sending
// succeeds -- this endpoint must never reveal what the registered address
// is or whether an account exists.
// Returns the currently configured authenticator secret (or null if
// TOTP_SECRET hasn't been set on this deployment), so the Owner Settings
// screen can show it for setting up a new device. Note: like every other
// endpoint in this API, there is no server-side login check here -- anyone
// with the API URL can call this, consistent with the rest of this app's
// security model (see README).
app.get('/api/totp-secret', (req, res) => {
  res.json({ secret: TOTP_SECRET || null });
});

app.post('/api/reset-pin/request', async (req, res) => {
  const generic = { ok: true };
  const raw = (req.body && (req.body.identifier || req.body.email) || '').trim();
  if (!raw) return res.json(generic);

  try {
    const rows = await sql`SELECT value FROM app_data WHERE key = 'settings'`;
    const settings = rows.length ? rows[0].value : {};
    const registeredEmail = (settings.ownerEmail || '').trim().toLowerCase();
    const registeredPhone = (settings.ownerPhone || '').replace(/\D/g, '');

    const inputLower = raw.toLowerCase();
    const inputDigits = raw.replace(/\D/g, '');

    const matchedEmail = !!(registeredEmail && registeredEmail === inputLower);
    const matchedPhone = !!(registeredPhone && inputDigits && registeredPhone ===

    if ((!matchedEmail && !matchedPhone) || !APP_URL) {
      if (!APP_URL) console.error('Reset requested but APP_URL is not set.');
```

```

const expiresAt = new Date(Date.now() + RESET_TOKEN_TTL_MS).toISOString();
await sql`
  INSERT INTO pin_resets (token, role, expires_at)
  VALUES (${token}, 'owner', ${expiresAt})
`;

const link = `${APP_URL}/#reset-pin/${token}`;

// Email and SMS are both best-effort bonuses, not requirements -- if
// either isn't configured or fails to send, the link is still returned
// directly below so the Owner (who just proved they know a registered
// email or phone) can open it themselves or share it via WhatsApp.
if (matchedEmail && mailer) {
  mailer.sendMail({
    from: `GreenLine Cargo <${GMAIL_USER}>`,
    to: registeredEmail,
    subject: 'Reset your GreenLine Cargo Owner PIN',
    text: `A reset was requested for your GreenLine Cargo Owner PIN.\n\nOpen
    html: `<p>A reset was requested for your GreenLine Cargo Owner PIN.</p>
      <p><a href="${link}">Click here to set a new PIN</a> (expires in 3
      <p>If you didn't request this, you can safely ignore this email.</
  }).catch(e => console.error('reset-pin email send failed (link is still ret
} else if (matchedEmail && !mailer) {
  console.error('Reset matched by email but GMAIL_USER/GMAIL_APP_PASSWORD are
}

if (matchedPhone && smsClient && TWILIO_PHONE_NUMBER) {
  smsClient.messages.create({
    body: `GreenLine Cargo: reset your Owner PIN here (expires in 30 min): ${
    from: TWILIO_PHONE_NUMBER,
    to: settings.ownerPhone,
  }).catch(e => console.error('reset-pin SMS send failed (link is still retur
} else if (matchedPhone && (!smsClient || !TWILIO_PHONE_NUMBER)) {
  console.error('Reset matched by phone but TWILIO_ACCOUNT_SID/TWILIO_AUTH_TO
}

res.json({ ok: true, link });
} catch (e) {
  console.error(e);
  res.json(generic); // never leak failure details to the client
}
});

// Verify an authenticator-app code. If it matches TOTP_SECRET, issue a
// reset token directly (no email/SMS round-trip needed) -- this path works
// even if Gmail/Twilio are misconfigured or unreachable, since it's fully
// self-contained on this server.

```

```

app.post('/api/reset-pin/totp-verify', async (req, res) => {
  const code = (req.body && req.body.code || '').trim();
  const boatId = (req.body && req.body.boatId || '').trim();
  if (!code || !/^\\d{6}$/.test(code)) {
    return res.status(400).json({ ok: false, error: 'Enter the 6-digit code from
  }
  try {
    // Each organization got its own totp_secret at signup. If a boatId is
    // given, use THAT organization's secret; otherwise fall back to the
    // single legacy TOTP_SECRET env var (kept only for backward safety).
    let secret = TOTP_SECRET;
    let ownerBoatId = boatId || null;
    if (boatId) {
      const rows = await sql`
        SELECT o.totp_secret FROM boats b
        JOIN organizations o ON o.id = b.organization_id
        WHERE b.id = ${boatId}
      `;
      if (rows.length && rows[0].totp_secret) secret = rows[0].totp_secret;
    }
    if (!secret) {
      return res.status(400).json({ ok: false, error: 'Authenticator reset is not
    }
    const valid = authenticator.verify({ token: code, secret });
    if (!valid) {
      return res.status(400).json({ ok: false, error: 'That code is incorrect or
    }

    await sql`DELETE FROM pin_resets WHERE expires_at < now() - interval '1 day'`

    const token = crypto.randomBytes(24).toString('hex');
    const expiresAt = new Date(Date.now() + RESET_TOKEN_TTL_MS).toISOString();
    await sql`
      INSERT INTO pin_resets (token, role, expires_at, boat_id)
      VALUES (${token}, 'owner', ${expiresAt}, ${ownerBoatId})
    `;

    res.json({ ok: true, token });
  } catch (e) {
    console.error(e);
    res.status(500).json({ ok: false, error: 'Something went wrong. Try again.' })
  }
});

// Confirm a reset: consumes the token and sets the new Owner PIN, scoped to
// whichever boat the token was issued for (falls back to the legacy
// unscoped settings row only for very old tokens with no boat_id, which

```

```

// will no longer occur going forward).
app.post('/api/reset-pin/confirm', async (req, res) => {
  const { token, newPin } = req.body || {};
  if (!token || !newPin || !/^d{4,6}$/.test(String(newPin))) {
    return res.status(400).json({ ok: false, error: 'Enter a valid 4\u20136 digit'
  }
  try {
    const rows = await sql`SELECT * FROM pin_resets WHERE token = ${token}`;
    const record = rows[0];
    if (!record || record.used || new Date(record.expires_at) < new Date()) {
      return res.status(400).json({ ok: false, error: 'That link is invalid or ex
    }
    if (!record.boat_id) {
      return res.status(400).json({ ok: false, error: 'This reset link is outdate
    }

    const settingsRows = await sql`SELECT value FROM app_data WHERE boat_id = ${r
    const settings = settingsRows.length ? settingsRows[0].value : {};
    settings.ownerPin = String(newPin);

    await sql`
      INSERT INTO app_data (boat_id, key, value, updated_at)
      VALUES (${record.boat_id}, 'settings', ${JSON.stringify(settings)}::jsonb,
      ON CONFLICT (boat_id, key) DO UPDATE SET value = ${JSON.stringify(settings)
    `;
    await sql`UPDATE pin_resets SET used = true WHERE token = ${token}`;

    res.json({ ok: true });
  } catch (e) {
    console.error(e);
    res.status(500).json({ ok: false, error: 'Something went wrong. Try again.' }
  }
});

// --- Super Admin dashboard (Phase 1 of multi-tenant rebuild) -----
// ADMIN_USERNAME / ADMIN_PASSWORD are set once as Render env vars, known
// only to you. There's no session system here (consistent with the rest of
// this API's low-auth model) -- the admin dashboard resends both values with
// every request, and each request is checked against these env vars fresh.
// This is fine for a single admin user; it would need a real session/JWT
// layer before adding more admin accounts.
const ADMIN_USERNAME = process.env.ADMIN_USERNAME;
const ADMIN_PASSWORD = process.env.ADMIN_PASSWORD;

function checkAdmin(req) {
  const u = (req.body && req.body.username) || (req.query && req.query.username)
  const p = (req.body && req.body.passkey) || (req.query && req.query.passkey) ||

```

```

    return !! (ADMIN_USERNAME && ADMIN_PASSWORD && u === ADMIN_USERNAME && p === ADM
}
function requireAdmin(req, res, next){
  if(!ADMIN_USERNAME || !ADMIN_PASSWORD){
    return res.status(503).json({ ok:false, error:'Admin login is not configured
  }
  if(!checkAdmin(req)) return res.status(401).json({ ok:false, error:'Invalid adm
  next();
}

app.post('/api/admin/login', (req, res) => {
  if(!ADMIN_USERNAME || !ADMIN_PASSWORD){
    return res.status(503).json({ ok:false, error:'Admin login is not configured
  }
  if(checkAdmin(req)) res.json({ ok:true });
  else res.status(401).json({ ok:false, error:'Incorrect username or passkey.' });
});

// List every organization (owner) with their boats attached, for the
// Super Admin overview screen.
app.get('/api/admin/organizations', requireAdmin, async (req, res) => {
  try{
    const orgs = await sql`SELECT id, boat_name, owner_name, contact_number, gmai
    const boats = await sql`SELECT id, organization_id, name, is_primary, status,
    const withBoats = orgs.map(o => ({
      ...o,
      boats: boats.filter(b => b.organization_id === o.id),
    }));
    res.json({ ok:true, organizations: withBoats });
  }catch(e){
    console.error(e);
    res.status(500).json({ ok:false, error:'Could not load organizations.' });
  }
});

// List boat requests (pending additional-boat approvals), newest first.
app.get('/api/admin/boat-requests', requireAdmin, async (req, res) => {
  try{
    const rows = await sql`
      SELECT br.id, br.organization_id, br.requested_boat_name, br.payment_screen
        br.status, br.admin_note, br.created_at, br.reviewed_at,
        o.boat_name AS org_boat_name, o.owner_name
      FROM boat_requests br
      JOIN organizations o ON o.id = br.organization_id
      ORDER BY br.created_at DESC
    `;
    res.json({ ok:true, requests: rows });
  }
}

```

```

    }catch(e){
      console.error(e);
      res.status(500).json({ ok:false, error:'Could not load boat requests.' });
    }
  });

// Approve a pending boat request: creates the new boat row and marks the
// request approved. This is the step you take after confirming the
// transfer landed in your account.
app.post('/api/admin/boat-requests/:id/approve', requireAdmin, async (req, res) =
  try{
    const rows = await sql`SELECT * FROM boat_requests WHERE id = ${req.params.id}
    const request = rows[0];
    if(!request) return res.status(404).json({ ok:false, error:'Request not found
    if(request.status !== 'pending') return res.status(400).json({ ok:false, erro

    const boatId = crypto.randomBytes(8).toString('hex');
    await sql`
      INSERT INTO boats (id, organization_id, name, is_primary, status)
      VALUES (${boatId}, ${request.organization_id}, ${request.requested_boat_nam
    `;

    // Pre-fill this new boat's Payment Details from the org's own bank
    // info (set at signup) -- editable separately per boat from here on.
    const orgRows = await sql`SELECT bank_account_name, bank_account_number FROM
    const org = orgRows[0];
    if(org && (org.bank_account_name || org.bank_account_number)){
      const initialSettings = { bankAccountName: org.bank_account_name || '', ban
      await sql`
        INSERT INTO app_data (boat_id, key, value, updated_at)
        VALUES (${boatId}, 'settings', ${JSON.stringify(initialSettings)}::jsonb,
        ON CONFLICT (boat_id, key) DO UPDATE SET value = ${JSON.stringify(initial
      `;
    }

    // The screenshot is returned once in this response (so the admin can
    // download it), then permanently cleared from the database -- it's
    // sensitive banking info and shouldn't be retained after approval.
    await sql`UPDATE boat_requests SET status = 'approved', reviewed_at = now(),
    res.json({ ok:true, boatId, paymentScreenshot: request.payment_screenshot ||
  }catch(e){
    console.error(e);
    res.status(500).json({ ok:false, error:'Could not approve this request.' });
  }
});

app.post('/api/admin/boat-requests/:id/reject', requireAdmin, async (req, res) =>

```

```

    try{
      const note = (req.body && req.body.note) || '';
      await sql`UPDATE boat_requests SET status = 'rejected', admin_note = ${note},
      res.json({ ok:true });
    }catch(e){
      console.error(e);
      res.status(500).json({ ok:false, error:'Could not reject this request.' });
    }
  });

// Test-only helper for this phase: lets you (or me, during testing) create
// a boat request without the owner-facing request UI existing yet -- that
// UI is later-phase work. Remove or restrict this once real owner signups
// exist.
app.post('/api/admin/boat-requests/seed', requireAdmin, async (req, res) => {
  try{
    const { organizationId, requestedBoatName, paymentScreenshot } = req.body ||
    if(!organizationId || !requestedBoatName) return res.status(400).json({ ok:fa
    const id = crypto.randomBytes(8).toString('hex');
    await sql`
      INSERT INTO boat_requests (id, organization_id, requested_boat_name, paymen
      VALUES (${id}, ${organizationId}, ${requestedBoatName}, ${paymentScreenshot
      `;
    res.json({ ok:true, id });
  }catch(e){
    console.error(e);
    res.status(500).json({ ok:false, error:'Could not create request.' });
  }
});

// --- Owner signup / org login (Phase 2 of the multi-tenant rebuild) -----
// Passkeys are hashed with scrypt (Node's built-in crypto, no extra
// dependency) -- never stored or compared as plain text.
function hashPasskey(passkey){
  const salt = crypto.randomBytes(16).toString('hex');
  const hash = crypto.scryptSync(String(passkey), salt, 64).toString('hex');
  return `${salt}:${hash}`;
}

function verifyPasskey(passkey, stored){
  if(!stored || !stored.includes(':')) return false;
  const [salt, hash] = stored.split(':');
  const check = crypto.scryptSync(String(passkey), salt, 64).toString('hex');
  try{ return crypto.timingSafeEqual(Buffer.from(hash,'hex'), Buffer.from(check,'
  catch(e){ return false; }
}

// Create a new owner + their one free boat. Returns the TOTP secret once,

```

```

// in the response, so the front-end can show it to the owner for adding to
// an authenticator app -- it is never returned again after this call.
app.post('/api/signup', async (req, res) => {
  try{
    const b = req.body || {};
    const required = ['boatName', 'ownerName', 'contactNumber', 'mobile', 'passkey'];
    for(const f of required){
      if(!b[f] || !String(b[f]).trim()) return res.status(400).json({ ok:false, e
    }
    if(String(b.passkey).length < 6) return res.status(400).json({ ok:false, erro

    // The first boat's name doubles as the owner's login username, so it
    // has to be unique across every organization -- otherwise two owners
    // could collide and neither could log in reliably.
    const existing = await sql`SELECT id FROM organizations WHERE lower(boat_name
    if(existing.length > 0) return res.status(409).json({ ok:false, error:'That b

    const orgId = crypto.randomBytes(8).toString('hex');
    const totpSecret = authenticator.generateSecret();
    const passkeyHash = hashPasskey(b.passkey);

    await sql`
      INSERT INTO organizations (
        id, boat_name, owner_name, contact_number, gmail, mobile, passkey_hash,
        bank_account_name, bank_account_number, tracking_link, viber_link,
        social_links, routes, totp_secret
      ) VALUES (
        ${orgId}, ${b.boatName}, ${b.ownerName}, ${b.contactNumber}, ${b.gmail ||
        ${b.bankAccountName || null}, ${b.bankAccountNumber || null}, ${b.trackin
        ${JSON.stringify(b.socialLinks || [])}::jsonb, ${JSON.stringify(b.routes
      )
    `;

    const boatId = crypto.randomBytes(8).toString('hex');
    await sql`
      INSERT INTO boats (id, organization_id, name, is_primary, status)
      VALUES (${boatId}, ${orgId}, ${b.boatName}, true, 'active')
    `;

    // Pre-fill this boat's own Payment Details from what was entered at
    // signup -- it stays editable separately per boat from here on, this
    // just saves re-typing it the first time.
    if(b.bankAccountName || b.bankAccountNumber){
      const initialSettings = { bankAccountName: b.bankAccountName || '', bankAcc
      await sql`
        INSERT INTO app_data (boat_id, key, value, updated_at)
        VALUES (${boatId}, 'settings', ${JSON.stringify(initialSettings)}::jsonb,

```

```

        ON CONFLICT (boat_id, key) DO UPDATE SET value = ${JSON.stringify(initial
    `;
}

    res.json({ ok:true, organizationId: orgId, boatId, totpSecret });
} catch(e) {
    console.error(e);
    res.status(500).json({ ok:false, error:'Could not create your account. Try ag
}
});

// Owner login: identify by boat name (case-insensitive) + passkey.
// "Forgot Password" for the #org Owner Login screen. Verifies the code
// against that organization's own authenticator secret (set at signup),
// then issues a token scoped to resetting THAT organization's login
// passkey -- distinct from the older per-boat ownerPin reset flow.
app.post('/api/org-reset/totp-verify', async (req, res) => {
    const boatName = (req.body && req.body.boatName || '').trim();
    const code = (req.body && req.body.code || '').trim();
    const generic = { ok: false, error: 'Incorrect boat name or code.' };
    if (!boatName || !code || !/^\d{6}$/.test(code)) return res.status(400).json(generic);
    try {
        const rows = await sql`SELECT id, totp_secret FROM organizations WHERE lower(
        const org = rows[0];
        if (!org || !org.totp_secret) return res.status(400).json(generic);
        const valid = authenticator.verify({ token: code, secret: org.totp_secret });
        if (!valid) return res.status(400).json(generic);

        await sql`DELETE FROM pin_resets WHERE expires_at < now() - interval '1 day'`

        const token = crypto.randomBytes(24).toString('hex');
        const expiresAt = new Date(Date.now() + RESET_TOKEN_TTL_MS).toISOString();
        await sql`
            INSERT INTO pin_resets (token, role, expires_at, organization_id)
            VALUES (${token}, 'org-owner', ${expiresAt}, ${org.id})
        `;
        res.json({ ok: true, token });
    } catch (e) {
        console.error(e);
        res.status(500).json({ ok: false, error: 'Something went wrong. Try again.' });
    }
});

app.post('/api/org-reset/confirm', async (req, res) => {
    const { token, newPasskey } = req.body || {};
    if (!token || !newPasskey || String(newPasskey).length < 6) {
        return res.status(400).json({ ok: false, error: 'Password must be at least 6

```

```

    }
    try {
      const rows = await sql`SELECT * FROM pin_resets WHERE token = ${token}`;
      const record = rows[0];
      if (!record || record.used || new Date(record.expires_at) < new Date() || !re
        return res.status(400).json({ ok: false, error: 'That link is invalid or ex
      }
      const passkeyHash = hashPasskey(newPasskey);
      await sql`UPDATE organizations SET passkey_hash = ${passkeyHash} WHERE id = $
      await sql`UPDATE pin_resets SET used = true WHERE token = ${token}`;
      res.json({ ok: true });
    } catch (e) {
      console.error(e);
      res.status(500).json({ ok: false, error: 'Something went wrong. Try again.' }
    }
  });
}

```

```

app.post('/api/org-login', async (req, res) => {
  try{
    const { boatName, passkey } = req.body || {};
    if(!boatName || !passkey) return res.status(400).json({ ok:false, error:'User
    const rows = await sql`SELECT * FROM organizations WHERE lower(boat_name) = 1
    const org = rows[0];
    if(!org || !verifyPasskey(passkey, org.passkey_hash)){
      return res.status(401).json({ ok:false, error:'Incorrect username or PIN.'
    }
    const boats = await sql`SELECT id, name, is_primary, status FROM boats WHERE
    res.json({
      ok:true,
      organization: {
        id: org.id, boatName: org.boat_name, ownerName: org.owner_name,
        contactNumber: org.contact_number, gmail: org.gmail, mobile: org.mobile,
        bankAccountName: org.bank_account_name, bankAccountNumber: org.bank_accou
        trackingLink: org.tracking_link, viberLink: org.viber_link,
        socialLinks: org.social_links, routes: org.routes,
      },
      boats,
    });
  }catch(e){
    console.error(e);
    res.status(500).json({ ok:false, error:'Something went wrong. Try again.' });
  }
});

```

```

// Owner requests an additional boat. Re-verifies the org's own passkey so
// this can't be called by anyone who merely knows the organizationId.
app.post('/api/boat-requests', async (req, res) => {

```

```

try{
  const { organizationId, passkey, requestedBoatName, paymentScreenshot } = req
  if(!organizationId || !passkey || !requestedBoatName){
    return res.status(400).json({ ok:false, error:'Missing required fields.' })
  }
  const rows = await sql`SELECT passkey_hash FROM organizations WHERE id = ${org}`
  const org = rows[0];
  if(!org || !verifyPasskey(passkey, org.passkey_hash)){
    return res.status(401).json({ ok:false, error:'Could not verify your account' })
  }
  const id = crypto.randomBytes(8).toString('hex');
  await sql`
    INSERT INTO boat_requests (id, organization_id, requested_boat_name, payment_screenshot)
    VALUES (${id}, ${organizationId}, ${requestedBoatName}, ${paymentScreenshot})
  `;
  res.json({ ok:true, id });
}catch(e){
  console.error(e);
  res.status(500).json({ ok:false, error:'Could not submit your request. Try again' });
});

const port = process.env.PORT || 3000;
app.listen(port, () => console.log(`Cargo API listening on :${port}`));

```